Handwritten-Digit Recognition on an Arduino Uno

A step-by-step team plan: porting a neural network from an FPGA to an ATmega328P with a 7-segment display

Reference project: https://github.com/Si7li/MNIST-Neural-Network-On-FPGA. Contents: 1 Feasibility · 2 Team & ways of working · 3 Materials · 4 Tech stack · 5 Architecture · 6 Step-by-step plan · **7 Timeline and gates (new)** · **8 How to do each phase (new)** · 9 Training the model · 10 Putting the model on the Uno · 11 Unit tests & testing · 12 Docker · 13 Pitfalls · 14 Stretch goals · 15 Resources · Appendices A–D.

> **What is new in v2.** Section 7 says which part must be finished before which, using numbered **gates** (G0–G9) instead of dates. Section 8 explains how to do every phase in plain steps, with a small analogy, a code sketch only where the step is tricky, and a gate check. v1's Sections 7–13 are now 9–15 and every cross-reference has been updated. Two content fixes are marked **(v2)**: the scikit-learn pixel range (Section 9) and a reproducibility caveat (Sections 8 and 13). Appendix D is a blank gate log.

1 · Is it possible? Yes.

The FPGA project is small: a 64 → 32 → 10 network (8×8 digit images in, 10 scores out) with 2,368 int8 weights. That is about 2.4 KB. The Uno has 32 KB of flash and runs at 16 MHz. The Uno cannot do the maths in parallel, but it does not need to.

> **Analogy.** The FPGA is 32 cooks chopping at the same time. The Uno is one cook doing all ~2,400 chops one after another. The cook is far slower (milliseconds instead of microseconds), but the recipe is so short that a human still sees the answer instantly.

	FPGA project (Nexys 3)	This plan (Arduino Uno)
Compute	32 parallel multiply-accumulate units, 99 clock cycles	One CPU, a sequential loop of ~2,400 multiply-adds
Time per digit	≈ 8 µs (from the repo)	estimated 5–15 ms; we will measure it
Weight storage	ROM built into the logic	PROGMEM in flash (≈ 2.5 KB of 32 KB)
Working memory	registers	≈ 100–400 B of 2 KB SRAM
Language	VHDL	C (avr-gcc)
Input	UART from a PC drawing app	USB serial from a PC drawing app
Output	7-segment display + LEDs	single-digit 7-segment display

Dry run already done (in my sandbox, not on a real board). I trained a 64 → 32 → 10 network on the scikit-learn digits data, quantized it to int8 with int32 accumulators, and wrote the C inference core. Results on 360 held-out images: float accuracy 96.7%, int8 accuracy 96.7%; the C code matched the Python integer model on 360 / 360 images (identical scores). Compiled for the ATmega328P without the Arduino core it needs 3,094 B flash (9.4%) and 104 B static RAM (5.1%). The reference repo reports 97.5%; the gap likely comes from a different training run and split.

Two things to check first. (1) I could read the repo's README but not its source files, so every detail below that borrows the repo's choices (pixel range, shift, rounding, UART frame) is verified in Phase 1. (2) The repo page shows no LICENSE file. Without a license, copying its code is not clearly allowed. Plan: re-implement the idea in our own code, credit the repo, and optionally ask the author (open an issue) for permission.

2 · Team and ways of working

Roles (one person can hold two; with 3 people, merge ML + DevOps, Firmware + Hardware, and App + QA; with 2 people see Section 7.6):

Role	Owns	Main deliverables
ML / quantization	Training, integer "golden model", export	train.py, quantize.py, export_arduino.py, accuracy report
Firmware	C inference core, serial parser, display driver	nn_core, protocol, seg7, main.cpp, native unit tests

Role	Owns	Main deliverables
Hardware bring-up	Wiring, segment test, timing measurements	wiring photo + pin map, bench test results
PC app	Drawing tool, serial client, batch sender	draw_digit.py, send_image.py, hil_test.py
QA / test	Test plan, hardware-in-the-loop runs, bug log	docs/test-report.md, pass/fail table
DevOps	Docker, Makefile, CI, README	Dockerfiles, docker-compose.yml, GitHub Actions

How the team works together

- **Contracts first.** Like agreeing on the plug shape before one person builds the plug and another the socket. On day 1, freeze: tensor shapes, fixed-point scheme, serial frame (Appendix A), pin map, repo layout.
- **One golden model.** The Python integer model is the truth. Every other piece (C code, board) must match it bit for bit.
- **Git rules.** Protected main, short feature branches, one reviewer per pull request, CI must be green to merge.
- **One card per step** of Section 6 on a shared board, each with its “done when” line and its gate from Section 7.
- **Hardware sharing.** Ideally one Uno + display rig per 1–2 people. With fewer rigs, host-side tests (Section 11) let everyone work without touching hardware.
- **15-minute sync** a few times a week: blocked? contract change? who needs the rig?

3 · Materials

Item	Qty	Purpose	Status
Arduino Uno R3 (ATmega328P)	1 per rig	Runs the network	Have
Single-digit 7-segment LED display (common cathode or common anode)	1 per rig	Shows the predicted digit	Have
USB-A to USB-B cable that carries data (not charge-only)	1 per rig	Power, flashing, serial link to the PC	Check
Resistors 220–330 Ω, ¼ W	8 per rig	One per segment (a–g) + decimal point. ≈ 9–14 mA per segment	Need
Breadboard (half-size is enough)	1 per rig	Wiring	Need
Male-to-male jumper wires	≈ 10 per rig	Wiring	Need
Multimeter with diode test (or 3 V coin cell + resistor)	1 per team	Find the common pin and whether the display is cathode or anode	Optional
Push button	1 per rig	Optional: run the built-in self-test (cycle 0–9)	Optional
PC: Linux / macOS / Windows, Python 3.11+, Docker, free USB port	1 each	Train, build, test, draw digits	Each member

Clone boards with a CH340 USB chip work too. They may need a driver on Windows/macOS and appear as /dev/ttyUSB0 on Linux instead of /dev/ttyACM0.

Wiring (direct drive, no driver chip needed)

Segment	a	b	c	d	e	f	g	dp (optional)	Common pin
Uno pin	D2	D3	D4	D5	D6	D7	D8	D9	GND (common cathode) or 5 V (common anode)

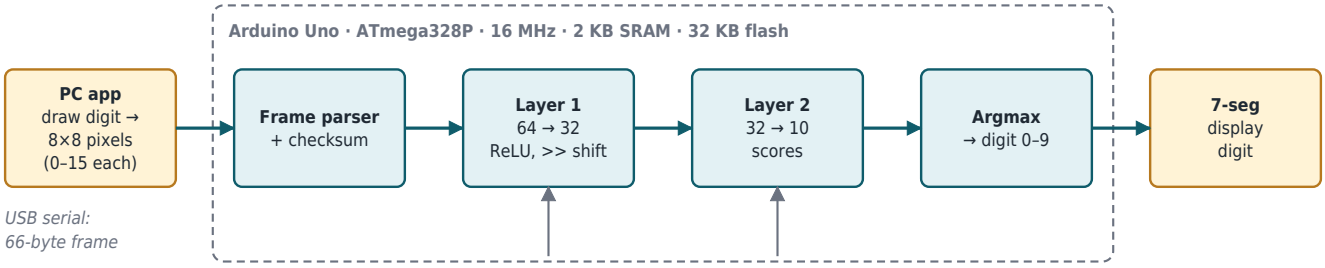
- Put a 220–330 Ω resistor in series with **every** segment pin. Never connect a segment straight to a pin.
- Avoid D0 and D1: they carry the USB serial link.
- Display pinouts differ between parts. Check its datasheet, or find the common pin and polarity with the multimeter diode test.
- Common cathode: a segment lights when its pin is HIGH. Common anode: it lights when LOW. The firmware has one flag for this.
- The decimal point can double as a “busy” light while a frame is received and computed.

4 · Tech stack

Layer	Tool	Why
Training	Python 3.11, PyTorch (CPU) + NumPy; scikit-learn for the dataset	Same approach as the repo. For a lighter Docker image, scikit-learn's MLPClassifier also works (my dry run used it).
Quantization + export	NumPy script (integer golden model) that writes weights.h and test vectors	One source of truth for weights and expected answers
Firmware	C/C++ on the Arduino core, avr-gcc, PlatformIO (atmelavr, board uno)	One tool for build, test, flash; works the same inside Docker
Inference core	Pure C file with no Arduino includes (nn_core.c)	Compiles on a PC for unit tests and on the AVR for the board
Unit tests	pytest (Python), Unity via PlatformIO native environment (C)	Fast tests with no hardware
PC app	Python, Tkinter, pyserial	Same stack as the repo's app
Flashing	avrdude (PlatformIO calls it for you)	Standard Uno uploader
Containers + CI	Docker, Docker Compose, Make, GitHub Actions	"git clone & make all" reproduces everything
Optional simulator	simavr	Run the firmware without a board (extra harness work, see Section 14)

Why not TensorFlow Lite Micro? It needs far more RAM than the Uno's 2 KB. A small hand-written C loop is the right tool here, just as the repo hand-built its hardware instead of using vendor IP.

5 · Architecture and data flow



Stop-and-wait protocol: the PC sends one frame, waits for the Uno's reply (digit, all 10 scores, time taken), then sends the next. Network: 64 inputs (4-bit pixels) → 32 hidden neurons with ReLU → 10 output scores → the biggest score is the digit.

6 · Step-by-step plan

After Phase 1, Phases 2, 3, 5 and 8 can run in parallel. Estimates are rough: about 2 weeks of calendar time for 3–5 people working part-time. **Section 7 turns this table into an exact order of gates** (what must be finished before what), and Section 8 explains how to do each phase.

Phase · owner	Tasks	Done when	Est.
P0 Kickoff All	• Create repo (layout in Section 12), task board, branch rules • Freeze contracts: shapes, fixed-point scheme, serial frame, pin map, file formats • Everyone installs Docker + Python	contracts.md merged; every member can run make test	0.5 d
P1 Study + spec ML + Firmware	• Read the repo README, python/train_quantize.py, app/draw_digit.py, the UART frame receiver in fpga/src • Record exact choices: pixel range, Q-format, shift, rounding, bias width, tie rule, frame format • Hand-compute one image end to end (pixels → hidden → scores → digit)	spec.md with the worked example, reviewed by 2 people	1 d
P2 Hardware bring-up Hardware	• Find common pin + polarity • Wire per Section 3 • Sketch: cycle digits 0–9, then all segments on	All segments and digits 0–9 look right; wiring photo + pin map committed	0.5–1 d

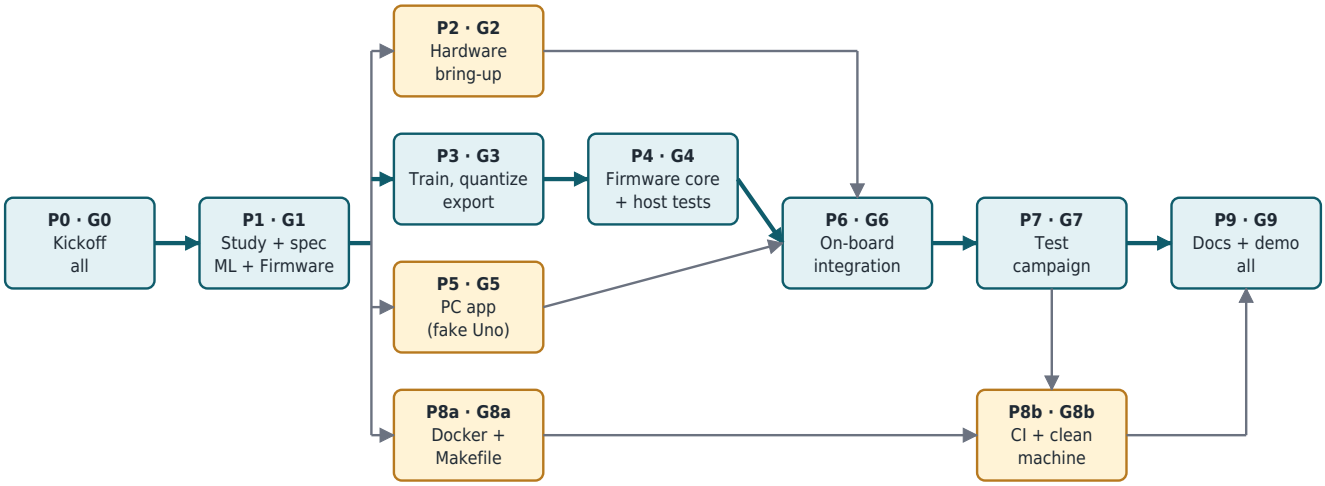
Phase · owner	Tasks	Done when	Est.
P3 Train, quantize, export ML	• train.py: fixed seed, stratified 80/20 split • quantize.py: integer golden model with the firmware's exact arithmetic • export_arduino.py: weights.h + golden vectors (inputs and expected scores) • Report: float vs int accuracy, confusion matrix, activation ranges (to pick the shift)	Int accuracy within 1 point of float; two exports give the same hash	1–2 d
P4 Firmware core + host tests Firmware	• nn_core (pure C, PROGMEM reads behind a macro), seg7, protocol parser • Unity tests (Section 11, levels 2–4) on the native environment	100% golden match on the host; tests green in CI	1–2 d
P5 PC app App (+ Firmware)	• Drawing canvas → 8×8, 0–15, same preprocessing as the training data • Serial client: wait ~2 s after opening the port (the Uno resets), stop-and-wait, timeout + retry • send_image.py (CLI) and hil_test.py (batch)	App shows the Uno's reply: digit, scores, microseconds	1–2 d
P6 On-board integration Firmware + Hardware	• Flash the board; run bench steps B1–B3 • Read the RAM/flash numbers from the build; measure time per digit	Known-answer images show the correct digit on the display	1 d
P7 Test campaign QA	• Bench steps B4–B7; log in docs/test-report.md • File bugs for any mismatch • Live trial with several people's handwriting	Hardware-in-the-loop: 0 mismatches; soak test clean; report written	1–2 d
P8 Docker + CI DevOps (from P1)	• Dockerfiles, compose file, Makefile; pin every version • Bake the PlatformIO toolchain into the image • GitHub Actions: tests + size budget + "regenerated weights equal committed weights" • Clean-machine test by someone new to the project	git clone && make all gives the same hash and .hex on a fresh PC	1–2 d
P9 Docs + demo All	• README with a ≤ 10-command quick start • Results table: accuracy, time per digit, flash/RAM; comparison with the FPGA • Demo GIF; credit the reference repo	A new person reproduces the project from the README alone	1 d

7 · Timeline and gates: what must be finished before what

There is no deadline, so this timeline has no dates. It has **gates**. A gate is a short checklist that proves a phase is really finished. The rule is simple: **a phase may start only when every gate it depends on has passed.**

> **Analogy.** A relay race. Runner 2 may not start until the baton has arrived in their hand, and "arrived" means the next runner (a second person) saw it arrive, not just that the first runner says they ran fast.

7.1 The map



An arrow means: the box on the left must pass its gate before the box on the right may start.

main chain (cannot be shared: each step needs the one before) side track (can run in parallel)

Gate numbers match phase numbers (G3 is the gate of P3). P8 is split in two: P8a builds the empty "container kitchen" early, P8b proves the whole project reproduces and can only finish at the end.

7.2 The gates, one by one

Gate	Phase	Must be finished first	Gate check (all must be true)	Unlocks
G0	P0 Kickoff	nothing	contracts.md merged · board has one card per phase · every member ran make test on their own machine	P1
G1	P1 Study + spec	G0	spec.md has every row of the choices table filled · worked example (one image by hand) reviewed by 2 people · licence decision written down	P2, P3, P5, P8a, and the parser + display part of P4
G2	P2 Hardware	G0 (the pin map is frozen there). Normally starts with the other tracks after G1; may start after G0 if the rig is idle	all 7 segments and digits 0–9 correct · wiring photo + pin map committed · polarity flag value written in contracts.md	P6
G3	P3 Train, quantize, export	G1	int accuracy within 1 point of float · two exports on the same machine give the same hash · NN_SHIFT chosen from data and recorded · weights.h + golden vectors committed	nn_core part of P4, batch sending in P5, P8b
G4	P4 Firmware core + host tests	G1 for parser and display; G3 for nn_core	100% golden match on the host · protocol and display tests green · the uno environment builds inside the size budget · *(CI check joins when P8b is done; until then "green" means make test on every machine)*	P6, P8b
G5	P5 PC app	G1 (frame + preprocessing); its batch sender needs the G3 vectors to be tested	app talks to a fake Uno and shows digit, scores, microseconds · frame builder and reply parser tests pass	P6
G6	P6 On-board integration	G2 and G4 and G5	bench steps B1–B3 pass in order · each of the ten known-answer images shows the right digit · flash, RAM and time per digit written down	P7

Gate	Phase	Must be finished first	Gate check (all must be true)	Unlocks
G7	P7 Test campaign	G6	B4 has 0 mismatches · B5 timing recorded · B6 soak clean · B7 and live trial hit rate logged · no open bug without a decision	P8b, P9
G8a	P8a Docker skeleton	G1	images build · make test runs inside a container on every member's machine · every version pinned	grows as G3 and G4 land
G8b	P8b CI + clean machine	G3, G4, G7, G8a	CI green on a pull request: tests, size budget, regenerated weights equal committed weights · a clean machine reproduces the same weights hash and .hex	P9
G9	P9 Docs + demo	G7 and G8b	a newcomer reproduces the project from the README alone · results table holds measured numbers · demo GIF made · reference repo credited	done

Effort, not deadlines. Along the main chain (P0 → P1 → P3 → P4 → P6 → P7 → P9) the sizes in Section 6 add up to roughly 6.5–9.5 person-days; the side tracks (P2, P5, P8) add about 2.5–5 more person-days that run alongside. The main chain cannot be split between people, so it sets the pace; extra people help only by finishing side tracks sooner.

7.3 Why this order

- **G1 before almost everything.** The spec fixes the plug shape (pixel range, shift, frame). Skipping it means rebuilding two or three pieces later.
- **G3 before nn_core.** You cannot grade a test without the answer key. The golden vectors from P3 are the answer key for the C code.
- **G2 and G5 before G6.** Do not debug wiring, firmware and the PC app at the same time. Prove each alone first, so any failure on the board has only one suspect.
- **G6 before G7.** Testing on a rig that is not proven gives noise, not results.
- **G7 before G9.** Do not write the user manual for a car that has not been driven.
- **Reopening a gate.** If a contract changes after its gate passed (for example the shift in G3), mark that gate red on the board and re-run the gate check of every gate downstream of it. A bug fix in P7 restarts the campaign from B4.

7.4 Work that is allowed early (it does not skip a gate)

After	Allowed to start	Still not allowed
G0	Hardware: identify the display, wire it, run the segment test (P2)	Anything that needs the spec (training, frame code)
G1	Parser + display driver + their Unity tests (first half of P4) · drawing canvas and preprocessing (P5) · Dockerfiles (P8a) · training script (P3)	nn_core (needs G3) · flashing a model
While waiting for G3	Test scaffolding that needs no weights: tie rule, worst-case maths on a small helper, protocol edge cases	Claiming any golden match
G4 + G5 + G2	Flash the board (P6)	Test campaign (needs G6)
G7	Results table, README, demo (P9)	Writing numbers that were not measured

7.5 Who works on what while a gate is closed

Waiting at a gate is a signal to **review** someone else's work, not to skip ahead. A reviewer who did not write the thing is also the person who signs the gate in Appendix D.

7.6 Splitting the phases between 2 or 3 people

You have 2–3 people and, today, one Uno + display rig. **Rule for one rig:** only one person holds it at a time, writes their name on the board card, and leaves it wired and working for the next person (photo of the wiring in docs/).

Phase	2 people (A = software side, B = hardware side)	3 people (A = ML + DevOps, B = Firmware + Hardware, C = App + QA)
P0	A + B together	A + B + C together
P1	A reads the Python side, B reads the UART/FPGA side, both fill spec.md	A and B as for 2 people; C reviews the spec as the newcomer
P2	B	B
P3	A	A

Phase	2 people (A = software side, B = hardware side)	3 people (A = ML + DevOps, B = Firmware + Hardware, C = App + QA)
P4	B (A reviews)	B (A reviews)
P5	A, using a fake Uno until P6	C, using a fake Uno until P6
P6	B holds the rig, A watches the PC side	B holds the rig, C watches the PC side
P7	A runs the bench steps on B's firmware work; B logs and fixes	C runs and logs; A and B fix
P8	A (B does the clean-machine test)	A (C does the clean-machine test)
P9	A + B	A + B + C

8 · How to do each phase

Every block has the same shape: **Analogy** (one line), **Needs** (the gates that must have passed), **Steps** (plain, in order), a **tricky bit** with code only where it is hard to get right, the **Gate check**, and **Watch out**. Code labelled "tested on a PC" was run in my sandbox (Python 3, gcc). Code labelled "sketch, not run" touches the Uno, Tkinter, Docker or GitHub Actions, which I could not run here; treat it as a starting point and fix it on the real rig.

P0 · Kickoff

Needs: nothing (this is the start).

> **Analogy.** Agreeing on the plug shape before one person builds the plug and another builds the socket.

Steps

1. One person creates the repo with the folder layout from Section 12 (empty folders, a README placeholder). Turn on branch protection for main: pull request + one reviewer.
2. Make the board. One card per phase P0–P9; copy the "done when" line from Section 6 and the gate name from Section 7 into each card.
3. Hold one 60–90 minute meeting and write **docs/contracts.md**. Five short paragraphs, one per contract: (a) shapes, 64 → 32 → 10, pixels are integers 0–15; (b) number scheme, int8 weights, int32 sums, one shift value *(a placeholder until G3, then frozen)*; (c) the serial frame from Appendix A; (d) pin map D2–D8 for a–g, D9 for the dot, plus the polarity flag; (e) repo layout and file names.
4. Agree the change rule: a contract changes only through a pull request that edits contracts.md and is approved by the owners of both sides of that contract.
5. Everybody installs Docker and Python 3.11. Add a tiny placeholder Makefile whose **make test** runs one trivial check inside a container, so everyone proves their machine works. The real targets arrive in P8.
6. Decide who flashes from the host. Docker Desktop on macOS/Windows cannot pass USB serial into a container, so on those machines the build runs in Docker but flashing runs on the host.

Gate check

- contracts.md merged
- board exists, one card per phase, each card shows its gate
- every member pasted a successful make test line on their card

Watch out

- Do not try to settle the shift value now. It depends on data from P3. The contract only says "one shift, named NN_SHIFT, frozen at G3".

P1 · Study + spec

Needs: G0.

> **Analogy.** Reading the original recipe before cooking your own version. You copy what makes it taste right, not the pan.

Steps

1. Split the reading. The ML person reads the README and the Python training/quantizing script. The firmware person reads the FPGA's UART receiver and the drawing app. File names are from v1 of this plan; confirm what the repo really contains and fix the names in this plan if they differ.
2. Create **docs/spec.md** with a "choices table", one row per choice: pixel range, weight format, shift after layer 1, rounding, bias width, tie rule, frame format, how drawings are preprocessed. Columns: *Repo's choice · Our choice · Same? · Where I found it*. Our default is whatever Appendices A and C say; change it only if matching the repo is cheap.
3. Write down the licence decision (re-implement in our own code, credit the repo, optionally open an issue asking the author). Keep reading notes as notes, never paste the repo's code into ours.
4. Hand-compute one image. Take one 8×8 test image, print it as 64 numbers, and work out **hidden neuron 0 only**: 64 products added to the bias, then ReLU, then shift, then clamp. Then do one output score from the 32 hidden values. Check your numbers with the ten-line script below.
5. Two people review spec.md. Each reviewer must be able to redo the worked example from the file alone.

The tricky bit

sketch, not run (uses w1, b1, SH from quantize.py in P3; do it on the repo's numbers in P1)

```
# neuron 0 for one test image: compare with your hand calculation
x = X_test[0]                          # 64 integers, 0..15
acc = int(b1[0])
for i in range(64):
    acc += int(x[i]) * int(w1[0][i])
print('sum before ReLU:', acc)
print('after ReLU, shift, clamp:', min(max(acc, 0) >> SH, 127))
```

Gate check

- every row of the choices table filled in
- worked example reviewed by two people
- licence decision written down

Watch out

- If the repo uses a different pixel range than 0–15, change contracts.md through a contract-change pull request before anybody moves on. See also the pixel-range note in Section 9.

P2 · Hardware bring-up

Needs: G0 (pin map). May start with the other tracks after G1.

> **Analogy.** Test the light bulb and the socket before blaming the electrician.

Steps

1. Unplug the Uno. Find out which kind of display you have. A common 10-pin display has five pins on top, five on the bottom; the middle pin of each row is usually the common pin (the two are joined inside). Use the multimeter diode test: red probe on the common pin and black on a segment pin lights the segment on a **common anode**; black on the common pin and red on a segment pin lights it on a **common cathode**. No multimeter: a 3 V coin cell in series with a 330 Ω resistor does the same job.
2. Write down which physical pin is segment a, b, c, d, e, f, g, the dot, and the common pin. Pinouts differ between parts, so do not trust a picture from another display.
3. Wire it: Uno D2…D8 to segments a…g, each through its own 220–330 Ω resistor; D9 to the dot (also through a resistor); common pin to GND (cathode) or 5 V (anode). Take a photo.
4. Upload the test sketch below. It shows 0–9, half a second each. If a segment is missing or a digit looks wrong, the wiring or the polarity flag is wrong, not the neural network.
5. Commit the wiring photo and the pin map to docs/, and write the polarity flag value into contracts.md.

The tricky bit

sketch, not run (needs the real display)

```
const uint8_t SEG[10] = {0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};  // Appendix B
const bool COMMON_ANODE = false;       // true for a common anode display

void show(uint8_t bits) {              // bit 0 = segment a ... bit 6 = segment g
  if (COMMON_ANODE) bits = ~bits;
  for (uint8_t s = 0; s < 7; s++) digitalWrite(2 + s, (bits >> s) & 1);
}
void setup() {
  for (uint8_t p = 2; p <= 9; p++) pinMode(p, OUTPUT);
  digitalWrite(9, COMMON_ANODE ? HIGH : LOW);   // dot off
}
void loop() { for (uint8_t d = 0; d < 10; d++) { show(SEG[d]); delay(500); } }
```

Gate check

- all 7 segments and digits 0–9 look right
- wiring photo + pin map committed
- polarity flag value recorded in contracts.md

Watch out

- A resistor on every segment, always. A bare pin can supply more current than a segment should take.
- Keep D0 and D1 free: they carry the USB serial link.

P3 · Train, quantize, export

Needs: G1.

> **Analogy.** Teach the chef in a big kitchen (decimal numbers), then write the recipe in whole numbers only so a tiny cook can follow it. The **golden model** is the printed recipe the tiny cook is checked against, step by step.

Steps

1. **train.py.** Load the scikit-learn digits, fix the random seed, split 80/20 stratified (360 test images), train 64 → 32 (ReLU) → 10 with Adam. Save the float weights to a file. Train twice with the same seed and compare hashes: they must match on the same machine.
2. **Pixel range (v2).** The dataset's pixels run 0–16, the contract says 0–15. Rescale once, in one shared function used by training and by the PC app: *x15 = round(x × 15 / 16)*. See the note in Section 9.
3. **Augmentation.** Add small one-pixel shifts to the training images (never to the 360 test images). It helps with hand-drawn digits in P7.
4. **quantize.py.** Write the golden model from Section 9: integers only, int32 sums. Print how many weights were clipped when squeezed into int8: it should be 0 or nearly 0. Check that integer accuracy is within 1 point of float.
5. **Choose the shift from data, not by habit.** Run the hidden layer over the training set, look at the largest values (for example the 99.9th percentile) before the clamp, and pick the shift that puts them near 100–127 without clipping often. In my dry run the hidden values peaked at 14, so a shift of 5 wasted most of the range. Save the histogram.
6. **export_arduino.py.** Write weights.h (arrays W1, B1, W2, B2 and NN_SHIFT) and the golden vectors for the host tests (360 inputs and their expected 10 scores each). Golden vectors go in the native test folder only; they are never flashed.
7. Export twice and compare SHA-256 hashes. Write the short report: float vs int accuracy, confusion matrix, histogram, chosen shift.

The tricky bit

tested on a PC: with random weights, the C core of Appendix C built from this header matched the Python golden model on 300 / 300 vectors, including all-15 and all-0 inputs

```
def c_array(name, a, ctype):
    body = ','.join(str(int(v)) for v in a.flatten())
    return f'const {ctype} {name}[{a.size}] PROGMEM = {{{body}}};\n'

hdr = ('#pragma once\n#include <stdint.h>\n#ifdef __AVR__\n#include <avr/pgmspace.h>\n#else\n'
       '#define PROGMEM\n#endif\n' + f'#define NN_SHIFT {SH}\n'
       + c_array('W1', w1, 'int8_t') + c_array('B1', b1, 'int16_t')
       + c_array('W2', w2, 'int8_t') + c_array('B2', b2, 'int16_t'))
open('firmware/include/weights.h', 'w').write(hdr)
```

Bias scale follows the shift. The second-layer bias is scaled by $(S / 2^{SH}) \times S$. With S = 64 and SH = 5 that is 2 × S, as in Section 9. If you change SH, change this factor too, or the answers drift. Biases need int16 (B1, B2), not int8, because they are multiplied by S.

Gate check

- integer accuracy within 1 point of float
- two exports give the same hash (same machine)
- NN_SHIFT chosen from the histogram and written into contracts.md
- weights.h and golden vectors committed; report reviewed by someone else

Watch out

- **(v2)** Float training can give slightly different weights on different CPUs. If two teammates' hashes differ, do not spend days chasing it: commit the float weights file once, make quantize and export deterministic from it, and treat re-training as a deliberate manual step (Section 12, reproducibility rules).

P4 · Firmware core + host tests

Needs: G1 for the parser and display part; G3 for the nn_core part.

> **Analogy.** Build and test the engine on a workbench stand (your PC) before lowering it into the car (the Uno).

Steps

1. **Part A, after G1.** Write the frame parser as a pure C state machine: it takes one byte at a time and says "nothing yet", "frame OK" or "bad". A tick function drops a half frame after 100 ms of silence. Write the 7-segment driver with the table from Appendix B, the polarity inversion, and a pin-writer function you can swap for a fake in tests.
2. **Part B, after G3.** Copy the inference core from Appendix C into lib/nn_core. Keep the PROGMEM reads behind the RD8 / RD16 macros so the same file compiles on the PC (plain pointers) and on the AVR (pgm_read_*).
3. Make the awkward parts testable on their own: pull out **argmax** as a small function (so a tie can be tested without special weights) and the **dot product** as a small function (so all-15 inputs against −128 weights can be tested without special weights).
4. Write the Unity tests for levels 2, 3 and 4 of Section 11. For level 2, loop over every golden vector and compare **all 10 scores**, not only the digit.
5. Run **pio test -e native**. Then run **pio run -e uno** (it compiles without a board) and read the Flash and RAM percentages against the budget in Section 11.
6. Write main.cpp last, from the skeleton in Section 10: read bytes as they arrive, compute only after a full frame, show the digit, send the reply.

The tricky bit

tested on a PC with five cases: good frame, bad checksum, junk before the header, half a frame then silence, a pixel above 15

```
proto_result_t proto_feed(proto_t *p, uint8_t b, uint32_t now_ms) {
    p->last_byte_ms = now_ms;
    switch (p->state) {
    case S_HEADER:                                  /* junk before 0xA5 is ignored */
        if (b == 0xA5) { p->state = S_PIXELS; p->count = 0; p->xor_sum = 0; }
        return PROTO_NONE;
    case S_PIXELS:
        if (b > 15) { reset(p); return PROTO_BAD; }  /* invalid pixel */
        p->pixels[p->count++] = b;  p->xor_sum ^= b;
        if (p->count == 64) p->state = S_CHECKSUM;
        return PROTO_NONE;
    case S_CHECKSUM: { uint8_t ok = (b == p->xor_sum); reset(p);
                       return ok ? PROTO_FRAME_OK : PROTO_BAD; }
    }
    return PROTO_NONE;
}
void proto_tick(proto_t *p, uint32_t now_ms) {         /* call from loop() */
    if (p->state != S_HEADER && (uint32_t)(now_ms - p->last_byte_ms) > 100) reset(p);
}
```

Gate check

- 100% golden match on the host (all 10 scores)
- protocol and display tests green
- pio run -e uno builds; flash ≤ 50%, static RAM ≤ 60%
- tests green with make test on every machine (and in CI once P8b adds it)

Watch out

- On AVR a plain int is 16 bits. Keep sums in int32_t and cast array indexes: (uint16_t)j * NN_IN + i.
- v1 listed “tests green in CI” here, but CI is built in P8. Until then, “green” means make test.

P5 · PC app

Needs: G1 (frame and preprocessing). Testing its batch sender needs the vectors from G3.

> **Analogy.** The app is the person at the counter writing the order slip. The slip format (the frame) is fixed, so the kitchen can read any slip.

Steps

1. **Preprocessing, first.** Put one function in a shared file: crop the drawing to the ink’s bounding box, pad to a square, centre it, shrink to 8×8 by **averaging**, scale to integers 0–15. In scikit-learn digits ink is a high number and background is 0; print one training image and check you match it.
2. **Canvas.** A Tkinter canvas about 280×280. Draw each stroke on the canvas *and* on a hidden Pillow image; preprocess the hidden image. Try thin and thick pens: mouse strokes are thicker than the dataset’s.
3. **Serial client.** Open the port at 115200, 8N1, then wait about 2 seconds (opening the port resets the Uno), clear the input buffer, send one frame, wait for the reply with a timeout (about 500 ms). On a timeout or on the single error byte 0x15, retry up to 3 times, then report. Stop-and-wait: never send the next frame before the reply.
4. **Two small tools.** send_image.py sends one image (from a file or a drawing) and prints digit, scores, microseconds. hil_test.py sends every golden vector, compares the digit *and all 10 scores*, and prints min / median / max time.
5. **A fake Uno.** Write a small Python program that answers like the real board, using the golden model (or canned bytes before G3). Connect it with a virtual serial pair (for example socat). Now the whole app can be tested with no hardware, and when P6 starts, the only new thing is the real board.

The tricky bit

tested on a PC against a fake reply; the serial port part (waiting 2 s, retries) is not run

```
def build_frame(pixels):                        # 66 bytes: header, 64 pixels, XOR
    assert len(pixels) == 64 and all(0 <= p <= 15 for p in pixels)
    x = 0
    for p in pixels: x ^= p
    return bytes([0xA5, *pixels, x])

def parse_reply(buf):                           # 47 bytes: 0x5A, digit, 10 x int32, uint32 us, XOR
    if buf == b'\x15': raise ValueError('Uno rejected the frame')
    if len(buf) != 47 or buf[0] != 0x5A: raise ValueError('wrong length or header')
    body = buf[1:46]
    x = 0
    for b in body: x ^= b
    if x != buf[46]: raise ValueError('reply checksum mismatch')
    return body[0], struct.unpack('<10i', body[1:41]), struct.unpack('<I', body[41:45])[0]
```

Gate check

- the app talks to the fake Uno and shows digit, scores and microseconds
- frame builder and reply parser tests pass
- one preprocessing function shared with training

Watch out

- Only one program can hold the serial port: close the app before flashing.

P6 · On-board integration

Needs: G2 (display works), G4 (firmware passes host tests), G5 (app works with the fake Uno).

> **Analogy.** The first time the engine goes into the car. The bench tests passed, so any new problem is in the connection between engine and car, which is a small place to look.

Steps

1. Build with **pio run -e uno**. Write the Flash and RAM numbers on the card.
2. Close every program that holds the port, then flash: **pio run -e uno -t upload --upload-port /dev/ttyACM0** (use **pio device list** to find the port).
3. **B1.** Power on: the display shows the dash while waiting. Then run the 0–9 cycle again to confirm the wiring survived.
4. **B2.** Serial echo: send one frame, expect the reply or the checksum, no framing errors. Only when B2 passes go on.
5. **B3.** Known answers: send one golden image per digit, and watch the display show the right digit each time.
6. Take a first timing: send the same image 100 times and note min, median, max microseconds. The formal timing is B5 in P7.

The tricky bit

If B3 fails but the host tests passed, bisect. Like finding which bulb in a string of lights is dead: test from the middle.

- Send an **all-zero image**. The scores then depend only on the biases, so a mismatch points to the bias reads from flash (PROGMEM).
- Send an image with **one pixel** set to 1 at position i. Scores then equal the biases plus one weight column, so a mismatch points to how weights are addressed.
- Compare **all 10 scores**, not just the digit: a wrong score that still leaves the same digit hides the bug.
- If even B2 fails: try 57600 or 250000 baud (115200 has about 2% timing error on a 16 MHz AVR), and check the cable carries data.

Gate check

- B1, B2, B3 pass, in that order
- ten known-answer images show the correct digit on the display
- flash, RAM and a first timing written down

Watch out

- Do not change firmware and wiring in the same step. One change, one test.

P7 · Test campaign

Needs: G6.

> **Analogy.** A second person tries to break it. The cook should not be the only one tasting the soup.

Steps

1. Whoever did not write the firmware runs the campaign (see Section 7.6). Make docs/test-report.md with one row per bench step: date, who ran it, result, evidence.
2. **B4.** Batch: run hil_test.py with all 360 golden vectors. Pass = 0 mismatches in digit *and* in all scores.
3. **B5.** Timing: 100 runs, record min / median / max microseconds. Compare with the estimate (5–15 ms) and with the repo's ≈ 8 µs; write the real figure.
4. **B6.** Soak: 1,000 frames back to back. No hangs, no resets, same answers each time.
5. **B7 and live trial.** Draw digits in the app. Several teammates each draw about 30 digits. Log hits and misses, and which digits get confused (for example 4 and 9). Agree the target beforehand, for example ≥ 80%.
6. For every mismatch, file a bug card and **save the exact 64-number input** so anybody can replay it.

Gate check

- B4: 0 mismatches
- B5 recorded
- B6 clean
- B7 and the live trial logged with a hit rate

- every open bug fixed, or listed with an explicit decision

Watch out

- A bug fix restarts the campaign from B4. If you patch the firmware, the earlier results no longer describe the firmware you are shipping.
- Hand-drawn digits scoring below the 97% test-set figure is a domain gap, not necessarily a code bug. Check preprocessing first (Section 13).

P8 · Docker + CI

Needs: P8a needs G1. P8b needs G3, G4, G7 and G8a.

> **Analogy.** Write the recipe card *and* pack a sealed ingredient box, so any kitchen cooks the same dish.

Steps

1. **P8a, early.** Write docker/Dockerfile.train (Python 3.11 plus pinned numpy, scikit-learn, pytest) and docker/Dockerfile.firmware (PlatformIO baked in). Write docker-compose.yml and a Makefile whose targets each call **docker compose run** for the matching service.
2. Pin everything: base image tag, a pip requirements file with exact versions, the PlatformIO version and the AVR platform version. Warm the toolchain cache inside the image so builds work offline.
3. Grow the Makefile as phases finish: make train after G3, make test after G4, make build, make hil. **make flash** stays Linux-only, because it needs the device passed in.
4. **P8b, at the end.** Add GitHub Actions with three checks: tests, the size budget (a script that parses the avr-size output), and “regenerated weights equal committed weights”.
5. **Clean-machine test.** Someone who has not touched the project clones it, follows the README, and compares the weights hash and the .hex hash with yours.

The tricky bit

sketch, not run (the regenerate-and-compare check)

```
make train                                  # CI step: retrain with fixed seeds
git diff --exit-code firmware/include/weights.h   # fails if the result differs from what is committed
```

Gate check

- G8a: images build, make test runs in a container on every machine, versions pinned
- G8b: CI green on a pull request; the clean machine reproduces the same hash and .hex

Watch out

- **(v2)** If the clean machine’s hash differs from yours only in the float weights, that is CPU maths, not a mistake. Compare the integer outputs instead (the golden vectors’ scores must be identical), or commit the float weights file.
- On NixOS, enable Docker and add yourself to the docker and dialout groups (Section 12).

P9 · Docs + demo

Needs: G7 and G8b.

> **Analogy.** Leave a map for the next explorer, drawn by someone who just walked the path.

Steps

1. Write the README quick start in ten commands or fewer: clone, make test, make build, flash, run the app.
2. Fill the results table with **measured** numbers only: float and integer accuracy, live hit rate, time per digit (median), flash and RAM used, and the comparison with the FPGA’s ≈ 8 µs.
3. Record the demo: a short screen-plus-phone video of drawing a digit and the display lighting up; convert to a GIF.
4. Credit the reference repo and state the licence decision from P1.
5. Final test: a person outside the team follows the README with no help.

Gate check

- a newcomer reproduces the project from the README alone
- results table uses measured numbers
- demo GIF in the README
- reference repo credited

9 · Training the model

Resources

- **Data:** scikit-learn digits, 1,797 images of 8×8 pixels. It ships inside scikit-learn, so there is no download and Docker builds stay offline and reproducible.
- **Compute:** any laptop CPU. The repo reports about 7 s of training. No GPU, no Colab.
- **Model:** 64 → 32 (ReLU) → 10 = 2,368 weights + 42 biases = 2,410 parameters.
- **Recipe:** fixed seeds, stratified 80/20 split (360 test images, same count as the repo), Adam optimizer, CPU only.
- **Hand-drawn digits are different from the dataset.** Mouse strokes are thicker and placed differently. Fix both sides: preprocess drawings like the dataset (crop, centre, average-pool to 8×8, scale to 0–15) and add small shifts as augmentation.

> **Pixel range (v2 correction).** v1 said the dataset's pixels are 0–15. They are **0–16** (17 levels; 9% of all pixels equal 16). The contract keeps 0–15 so that a pixel still fits in 4 bits and the frame is unchanged, so **rescale once**: x15 = round(x × 15 / 16), in the same shared function for training and for the app. I re-checked three ways on the same split (360 test images): (A) keep 0–16 and divide by 16: float 96.9%, int8 96.9%; (B) rescale to 0–15 and divide by 15: float 96.7%, int8 96.7%; (C) clip 16 down to 15: float 97.2%, int8 96.1%. One image is 0.28 points, so A and B are equal within noise; C loses some after quantizing. **Recommendation: B**, because it keeps Appendix A and every "0–15" in this plan correct. Phase 1 may still change this to match the repo.

Quantization scheme (proposal, from my dry run)

Weights become int8 with 6 fractional bits (scale 64). Pixels stay integers 0–15. Sums use int32. After layer 1: ReLU, shift right, clamp to 0–127. The digit is the first maximum score (NumPy's argmax rule).

*python/quantize.py (core of the golden model). If you change SH, the b2 factor becomes (S / 2**SH) * S.*

```
S, SH = 64, 5     # weight scale (Q2.6), hidden-layer shift
w1 = np.clip(np.rint(W1 / 15.0 * S), -128, 127).astype(np.int32)   # model trained on x/15
b1 = np.rint(b1_float * S).astype(np.int32)
w2 = np.clip(np.rint(W2 * S), -128, 127).astype(np.int32)
b2 = np.rint(b2_float * 2 * S).astype(np.int32)    # hidden scale is S / 2**SH = 2

def golden(x):                          # x: ints 0..15 (already rescaled), shape (N, 64)
    a1 = x @ w1.T + b1
    h = np.clip(np.maximum(a1, 0) >> SH, 0, 127)
    return h @ w2.T + b2                # int32 scores; digit = argmax
```

- **Why int32:** worst case layer 1 is 64 × 15 × 128 ≈ 123,000, far above the int16 limit of 32,767. On AVR a plain int is only 16 bits.
- **Tune the shift.** In my dry run the hidden values peaked at 14, so a shift of 5 wastes most of the 0–127 range, and layer-1 weights only spanned −5…+6 because the model was trained on normalised inputs. Accuracy still held, but pick the shift from the activation histogram (Phase 3), and match what the repo does (Phase 1).
- **Stretch idea:** full 28×28 MNIST (60k training / 10k test images, via torchvision or OpenML). A 784 → 16 → 10 net is ≈ 12.7 KB of weights and fits in flash, but 784 pixels do not fit comfortably in 2 KB of RAM, so layer 1 would accumulate pixels as they stream in. Expect lower accuracy than the 8×8 version; measure it.

10 · Putting the trained model on the Uno

The model is not "loaded" like a file. It is compiled into the firmware as constant arrays in flash. A new model means a new weights.h and a re-flash.

Steps

1. Export. `python python/export_arduino.py` writes firmware/include/weights.h (arrays W1[2048], B1[32], W2[320], B2[10] in PROGMEM) and the golden vectors for host tests only (not flashed).

2. Read weights the AVR way. Flash and RAM are separate address spaces on the Uno. Reading a PROGMEM array with a normal pointer returns garbage, so use pgm_read_byte / pgm_read_word and cast to a signed type.

3. Build. `pio run -e uno`. Check the printed Flash/RAM percentages against the budget in Section 11.

4. Flash. Close the PC app first (only one program can hold the serial port), then `pio run -e uno -t upload --upload-port /dev/ttyACM0`.

5. First light. The display shows "-" while waiting. Run the bench steps in Section 11.

Troubleshooting: permission denied → add yourself to the dialout group (uucp on Arch); wrong port → pio device list; upload hangs → wrong board or port busy.

platformio.ini

```
; firmware/platformio.ini
[env:uno]
platform = atmelavr         ; pin an exact version once the team picks one
board = uno
framework = arduino
monitor_speed = 115200

[env:native]                ; runs unit tests on the PC, no board needed
platform = native
test_framework = unity
```

main.cpp

```
// firmware/src/main.cpp (skeleton)
#include <Arduino.h>
extern "C" {
#include "nn_core.h"
}
#include "seg7.h"
#include "protocol.h"

static uint8_t img[NN_IN];
static int32_t scores[NN_OUT];

void setup() {
  seg7_init();                // pins D2..D8, common-anode flag lives in seg7.h
  seg7_show_dash();           // "-" means waiting
  Serial.begin(115200);
}

void loop() {
  if (!protocol_read_frame(img)) return;   // non-blocking; timeout resets the parser
  seg7_set_dp(true);                       // busy light
  uint32_t t0 = micros();
  uint8_t digit = nn_infer(img, scores);
  uint32_t dt = micros() - t0;
  seg7_show(digit);
  protocol_reply(digit, scores, dt);       // see Appendix A
}
```

11 · Unit tests and how to test

Test levels (bottom = fastest, no hardware)

Level	What it checks	Tool · where	Pass when
1 · Python unit	Weights fit in int8; shapes (32×64, 10×32); same seed gives the same weights.h hash; int accuracy within 1 point of float	pytest · Docker	All green
2 · C core (host)	ReLU/clamp edge cases; worst-case inputs do not overflow (all pixels 15 × weight −128); argmax tie picks the lowest index; every golden vector gives identical scores	Unity, PlatformIO native · Docker	0 mismatches
3 · Protocol	Good frame accepted; bad checksum rejected; truncated frame times out; garbage before the header resyncs; pixel above 15 handled as the spec says	Unity native · Docker	All green
4 · 7-segment driver	Table for 0–9 correct (Appendix B); common-anode output is the inverse; uses a mock pin writer	Unity native · Docker	All green
5 · Build budget	Parse the avr-size output of the build	Script in CI · Docker	Flash ≤ 50%, static RAM ≤ 60% (keeps stack headroom)
6 · On-board test (HIL)	hil_test.py sends all golden vectors over USB; compares digit and all 10 scores; records time per digit	Python · real board	100% match
7 · Simulator (optional)	Run the .elf in simavr with a small harness	simavr · Docker	Same as level 6
8 · Acceptance	Team members draw 30 digits live	PC app · real board	Team-agreed target, e.g. ≥ 80% (hand-drawn is below the 97% test-set figure)

Bench procedure on the board (do in order)

Step	Test	Pass when
B1	Power + display: cycle 0–9 every 500 ms, then all segments on	Every segment lights; digits look right
B2	Serial echo: PC sends a frame, Uno answers with the checksum	Reply matches; no framing errors
B3	Known answer: send one golden image per digit	Display shows the right digit each time
B4	Batch: hil_test.py with all 360 vectors	0 mismatches in digit and scores
B5	Timing: 100 runs, min / median / max microseconds	Recorded (my estimate: 5–15 ms)
B6	Soak: 1,000 frames back to back	No hangs, no resets, same answers
B7	Live demo: draw digits in the app	Answers shown on the display; note the hit rate

12 · Making it reproducible with Docker

Goal: on any machine, `git clone && make all` gives the same weights, the same .hex and the same test results.

What runs where

Task	Container?	Notes
Train, quantize, export	Yes (train image)	CPU only, seeds fixed
Unit tests (levels 1–5)	Yes	No hardware needed
Build the .hex	Yes (firmware image)	PlatformIO toolchain baked into the image
Flash + hardware tests	Yes on Linux, pass the device in	--device /dev/ttyACM0. Docker Desktop on macOS/Windows cannot pass USB serial: build in Docker, flash from the host (or use usbipd with WSL2)
Drawing app (Tkinter)	Run on the host	GUI forwarding from containers is fiddly. Docker still has the headless send_image.py

Suggested repo layout

```
arduino-mnist-nn/
├─ python/    train.py quantize.py export_arduino.py tests/
├─ firmware/  platformio.ini src/main.cpp include/weights.h (generated)
│            lib/nn_core lib/protocol lib/seg7 test/
├─ app/       draw_digit.py send_image.py
├─ tools/     hil_test.py size_check.py
├─ docker/    Dockerfile.train Dockerfile.firmware
├─ docs/      spec.md contracts.md wiring.jpg test-report.md
├─ docker-compose.yml  Makefile  README.md
```

Dockerfile (sketch, not yet run)

```
# docker/Dockerfile.firmware
FROM python:3.11-slim
RUN apt-get update && apt-get install -y --no-install-recommends gcc make git \
 && rm -rf /var/lib/apt/lists/*
RUN pip install --no-cache-dir platformio==<pinned-version>
WORKDIR /work
COPY firmware/platformio.ini firmware/
# warm-up: download the AVR toolchain into the image (builds then work offline)
RUN cd firmware && pio pkg install -e uno && pio pkg install -e native
```

docker-compose.yml (sketch, not yet run)

```
# docker-compose.yml (excerpt)
x-fw: &fw
  build: { context: ., dockerfile: docker/Dockerfile.firmware }
  volumes: [".:/work"]
services:
  train:
    build: { context: ., dockerfile: docker/Dockerfile.train }
    volumes: [".:/work"]
    command: make -C python all
  test:
    <<: *fw
    command: make test
  build:
    <<: *fw
    command: make -C firmware hex
  flash:                       # Linux hosts only
    <<: *fw
    devices: ["/dev/ttyACM0:/dev/ttyACM0"]
    command: make -C firmware upload
```

Make targets: make train · make test · make build · make flash · make hil · make all. Each one just calls `docker compose run` for the matching service.

Reproducibility rules

- Pin everything: base image tag, pip lock file, PlatformIO and platform versions. Fixed seeds, CPU-only training, deterministic mode.
- Commit the generated weights.h (a few KB) so firmware builds do not need training. CI retrains and fails if the result differs from the committed file. **(v2)** Float maths can differ slightly between CPUs; if CI and a laptop disagree, commit the float weights file too and make the quantize/export step deterministic from it.
- PlatformIO downloads toolchains at first build; baking them into the image avoids surprises.
- Final check: a teammate on a clean machine runs the README commands and compares the .hex hash.

NixOS hosts: enable Docker with `virtualisation.docker.enable = true;` and add your user to `extraGroups = [ "docker" "dialout" ];`. For the host-side drawing app: `nix-shell -p "python3.withPackages (ps: [ ps.tkinter ps.pyserial ])"`.

13 · Pitfalls to expect

Pitfall	What happens	Fix
int is 16-bit on AVR	Silent overflow, wrong digits	Use int32_t accumulators; test with worst-case inputs
Reading PROGMEM like normal memory	Garbage weights	pgm_read_byte / pgm_read_word, cast to int8_t / int16_t
Opening the serial port resets the Uno	First frame is lost	PC app waits ~2 s after opening the port
Serial receive buffer is 64 bytes; a frame is 66	Dropped bytes if the loop is busy	Read bytes as they arrive, compute only after the full frame; stop-and-wait from the PC
115200 baud has ≈ 2% error on a 16 MHz AVR	Usually fine; rarely garbled frames	Try 57600 or 250000 if errors appear
Python and C disagree by one	Shift, rounding or tie-rule mismatch	Golden vectors on every build; compare all 10 scores, not just the digit
Hand-drawn digits fail but test set passes	Domain gap, not a code bug	Same preprocessing as training data; augmentation
Two programs want the serial port	“Port busy” on flash or app start	Close the app before flashing
Common anode vs cathode swapped	Display shows inverted segments	One compile-time flag; test with B1 first
Too much current through a pin	Hot pins, damaged chip	Always use resistors (20 mA per pin is the recommended limit, 40 mA absolute max)
Copying code from an unlicensed repo	Unclear legal footing	Re-implement, credit, ask the author
(v2) Dataset pixels go 0–16, contract says 0–15	Parser rejects a 16, or training and app disagree	Rescale once in a shared function (Section 9)
(v2) Float training differs across CPUs	Weights hash differs on another laptop	Compare integer outputs, or commit the float weights file
(v2) Skipping a gate “to save time”	A bug found late forces re-testing everything after it	Section 7: gate first, then start; reopen gates when a contract changes

14 · Stretch goals

- **Speed:** loop over pixels and skip the zeros (most of an 8×8 digit is empty). Could give a 2–3× speed-up; measure it.
- **Full 28×28 MNIST** with a streaming first layer (Section 9).
- **Smaller or smarter weights:** 4-bit weights, quantization-aware training.
- **simavr in CI:** run the firmware in a simulator. Needs a small C harness that connects the simulated UART to the test script, so budget extra time.
- **Confidence:** blink the decimal point when the top two scores are close, or plot scores on the serial plotter.
- **Report:** a side-by-side table of FPGA vs Uno (time, resources, effort).

15 · Resources

Topic	Link
Reference project (read the README + python/ folder first)	https://github.com/Si7li/MNIST-Neural-Network-On-FPGA
scikit-learn digits dataset	https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html
PyTorch (CPU install)	https://pytorch.org/get-started/locally/
PlatformIO: Atmel AVR platform	https://docs.platformio.org/en/latest/platforms/atmelavr.html
PlatformIO: unit testing	https://docs.platformio.org/en/latest/advanced/unit-testing/index.html
Arduino Uno R3 hardware docs	https://docs.arduino.cc/hardware/uno-rev3/
avr-libc: data in program space (PROGMEM)	https://www.nongnu.org/avr-libc/user-manual/pgmspace.html
simavr (AVR simulator)	https://github.com/buserror/simavr
Integer-only quantization paper (Jacob et al., 2018)	https://arxiv.org/abs/1712.05877
Docker Compose	https://docs.docker.com/compose/

Appendix A · Serial protocol (proposal; confirm against the repo in Phase 1)

115200 baud, 8N1, stop-and-wait. All multi-byte numbers are little-endian.

Direction	Bytes	Layout
PC → Uno (request, 66 B)	1 + 64 + 1	0xA5 · 64 pixel bytes (0–15 each, row by row) · XOR of the 64 pixel bytes
Uno → PC (reply, 47 B)	1 + 1 + 40 + 4 + 1	0x5A · digit (0–9) · 10 × int32 scores · microseconds taken (uint32) · XOR of all bytes between header and checksum
Uno → PC (error, 1 B)	1	0x15 on a bad checksum or invalid pixel; the PC may resend

Time on the wire at 115200 baud: request ≈ 5.7 ms, reply ≈ 4.1 ms. The parser drops a half-received frame after 100 ms without a byte.

Appendix B · 7-segment table and display logic

Bit 0 = segment a … bit 6 = segment g. For common anode, invert the byte before writing it to the pins.

Digit	0	1	2	3	4	5	6	7	8	9	dash
Byte	0x3F	0x06	0x5B	0x4F	0x66	0x6D	0x7D	0x07	0x7F	0x6F	0x40
Lit	abcdef	bc	abdeg	abcdg	bcfg	acdfg	acdefg	abc	abcdefg	abcdfg	g

Appendix C · Inference core (tested)

This is the C core from my dry run. It compiled with gcc and avr-gcc 7.3 and matched the Python golden model on all 360 test vectors. In v2 I rebuilt it on a PC against a freshly exported weights.h (random weights, 300 vectors including all-15 and all-0) and again got identical scores. Treat it as a starting point; Phase 1 may change the shift, rounding or clamp to match the repo.

lib/nn_core/nn_core.c

```
#include "nn_core.h"
#include "weights.h"

#ifdef __AVR__
  #define RD8(p)  ((int8_t)pgm_read_byte(p))
  #define RD16(p) ((int16_t)pgm_read_word(p))
#else
  #define RD8(p)  (*(p))
  #define RD16(p) (*(p))
#endif

uint8_t nn_infer(const uint8_t *x, int32_t *scores_out) {
  uint8_t hid[NN_HID];
  for (uint8_t j = 0; j < NN_HID; j++) {
    int32_t acc = RD16(&B1[j]);                      /* int32: int is 16-bit on AVR */
    for (uint8_t i = 0; i < NN_IN; i++)
      acc += (int16_t)x[i] * RD8(&W1[(uint16_t)j * NN_IN + i]);
    if (acc < 0) acc = 0;                            /* ReLU */
    acc >>= NN_SHIFT;
    hid[j] = (acc > 127) ? 127 : (uint8_t)acc;       /* clamp */
  }
  uint8_t best = 0; int32_t best_s = 0;
  for (uint8_t k = 0; k < NN_OUT; k++) {
    int32_t acc = RD16(&B2[k]);
    for (uint8_t j = 0; j < NN_HID; j++)
      acc += (int16_t)hid[j] * RD8(&W2[(uint16_t)k * NN_HID + j]);
    if (scores_out) scores_out[k] = acc;
    if (k == 0 || acc > best_s) { best_s = acc; best = k; }   /* first max wins, like numpy */
  }
  return best;
}
```

Appendix D · Gate log (print this, or copy it to the board)

A gate is signed by someone who did **not** write the thing being checked. Write the evidence (a file path, a pull request number, a log line), not just a tick.

Gate	Passed on (date)	Checked by	Evidence	Reopened? (why)
G0 Kickoff				
G1 Study + spec				
G2 Hardware				
G3 Train + export				
G4 Firmware core				
G5 PC app				
G6 On-board				
G7 Test campaign				
G8a Docker skeleton				
G8b CI + clean machine				
G9 Docs + demo				